Grant Compliance Assistant — Build Plan

Taps for Rights (Water Care AI Hackathon · Challenge A6) · Design locked, pre-build · Scenario date: 30 June 2022

Purpose. An AI assistant that helps Water Care staff see what the funder requires, what is missing or inconsistent, and what is due — with every finding traceable to a document, section and page. Depth over breadth: one tool that works, answering the 25 test questions correctly with verified citations or correctly flagged conflicts. All data is synthetic and deliberately salted with errors for the tool to catch.

1. Non-negotiable principles

1. **Cite every finding** — document, version, section/clause, page (or sheet + cell range).
2. **Never invent** obligations, evidence, indicators, figures or results. If absent, say "not found in the provided documents" and name what was searched.
3. **Flag ambiguity, don't resolve it.** When sources disagree, show both, name the question, name the resolver (Water Care staff or GAC). Article 1.3 precedence is context only — it never closes a conflict.
4. **Track versions** — including which General Terms & Conditions apply (the version in force on the Effective Date, 21 June 2019).
5. **People decide.** The tool drafts and flags; it never submits, sends or certifies compliance.
6. **No hardcoded answers.** Nothing in product code encodes answers to the test questions. The pipeline discovers them.

2. Locked design decisions

#	Question	Decision
1	Architecture	**One Q&A core + dashboards as views.** Backend exposes APIs; finding store is the single source of truth; FE never calls a model.
2	Absent GTC/PIP/forms	**Known-absent registry.** Abstain with metadata (GTC = version in force 21 Jun 2019); do not fetch; `check_absent_source` gates it.
3	Models	**GPT-4.1-mini default, escalate on eval-fail** to GPT-4.1 / Gemini 2.5 Pro; pre-pin

the ~10 conflict/synthesis intents to strong.

4	Date engine	**Compute-vs-asserted.** Emit rule_date + asserted_date + mismatch flag; status vs AS_OF (30 Jun 2022).
5	Urgency (Q25)	**Rule-based score** (overdue > imminent > active violation > inconsistency); model writes only the "why".
6	Due dates	**Live on obligations;** findings reference them by id.
7	Retrieval	**Full-context, no vector DB;** dispatch map loads a minimal doc set per intent.
8	Budget numbers	**Percentages + rules only** for answers; dollars absent → abstain. Labelled test-data fixtures exercise checks; firewalled from findings.
9	Demo spine	**All 25,** as cached pipeline findings with verified citations + doc-preview highlights (not hardcoded); gold file stays separate for eval.

3. Architecture — one core, three tool views

The three "tools" are **surfaces over one shared pipeline**, not three codebases. Tools depend on core; core depends on nothing; no tool imports another tool. The pipeline (ingest → dispatch → checks → draft → validate → verify) lives once in `core` and is invoked by each tool.

Tool	What it is	Shares core?	Inputs	Owns Qs	Priority
2. Compliance assistant	Default view — Q&A + issues dashboard + reporting calendar	Yes	03, 04, 05, 06	Q1–4, 6–20, 22, 24, 25 (~20)	Build first & fullest
1. Proposal assistant	Second view — call-requirements checklist	Yes	01, 02, 05, 06	Q5, 21, 23-proposal	Build second (cheap, reuse)
3. Funding radar	Separate optional module — live 2026 web, no test data	No	fixed public URLs (file 00 §C)	none of the 25	Stretch — cut first

Module layout

```
core/                # the engine — owns the pipeline, no tool knowledge
  ingest/            # PDF/Excel -> units + locators
```

```
  registry.py         # doc registry incl. known-absent (GTC, PIP, forms)
  schema.py           # Finding, Evidence, SourceRef, Conflict, Obligation  <- the contract
  store.py            # finding store (cache-backed)
  dispatch.py         # intent -> docs -> checks -> tier map
  checks/             # deterministic oracles + validate()/escalate gate
  llm.py              # tiered client (mini/strong) + prompts
  verify.py           # quote verifier
tools/
  compliance/         # config {03,04,05,06} + views (obligations/calendar/issues) + routes
  proposal/           # config {01,02,05,06} + checklist view + routes
  radar/              # own data source; imports only core.schema + core.llm
api/                  # thin — mounts tool routes, shared POST /ask, GET /findings
```

Scaling lever is the schema, not the folders. `core/schema.py` (Finding + SourceRef) is the contract between core↔tools and backend↔FE. Keep it stable and additive; a Tool 4 is a new module + dispatch entries, core untouched. **Anti-pattern to avoid:** deterministic checks and prompts never migrate into a tool — shared checks (e.g. `check_cost_share`, used by Q22 and Q5) live once in `core/checks`.

API surface

- `POST /ask` — Q&A (runs dispatch → checks → draft → validate/escalate)
- `GET /obligations`, `GET /calendar`, `GET /issues` — compliance dashboards (queries over findings)
- `GET /checklist` — proposal: one row per call requirement vs proposal
- `GET /findings` / `GET /findings/{id}` — all findings / drill-down (quoted text + locator for doc-preview highlight)
- `GET /radar` — stretch

4. Runtime pipeline

```
staff input -> router(rules) -> load doc set -> deterministic checks -> LLM drafts finding
            -> quote verifier -> validate() -> [escalate if invalid] -> cited finding ->
store
```

Model & cost strategy

Provider: OpenRouter (no sponsor credits). Start **GPT-4.1-mini for every call**; run the 25-question gold eval; escalate only intents that fail. Pre-pin the conflict/synthesis intents to the strong tier (GPT-4.1 or Gemini 2.5 Pro). Corpus is ~50 pages, so cost is dominated by (a) tier and (b) not resending the whole corpus — the dispatch loads a minimal doc set per intent.

Escalation gate — validate()

A response is **valid** if it survives objective checks (no LLM-judge). Escalate when any fire: schema invalid / truncated; any quote fails verification; status/evidence inconsistent; contradicts a deterministic result; empty/refusal where a source existed. **Do not escalate** a consistent `conflict`/`not_found`, or answers whose content lives in a known-absent source.

```
answer = cheap_model(q, ctx)
reason = validate(answer, ctx)              # reuses quote-verifier + checks
if reason: answer = cheap_model(q, ctx, repair=reason);  reason = validate(answer, ctx)   # 1 retry, same model
if reason: answer = strong_model(q, ctx, repair=reason); reason = validate(answer, ctx)   # escalate once
if reason: answer.status = "needs_review"; flag_human(reason)
# stop, no loop
log(q, tier_used, reason)   # doubles as hallucination-rate + the pin list for eval pass 2
```

5. Dispatch map (intent → docs → checks → tier)

strong = pre-pinned (skip mini). Others start mini; the gate escalates on failure.

Intent	Docs (sections)	Checks	Tier	Qs
agreement_date	03: Art.3, 4, 12	check_dates	mini	1, 2
cost_share_read	03: Art.2, App.B§1.6	check_cost_share	mini	3, 6
cost_share_conflict	03: Art.2.2–2.4; 01: cost share p.5; 02: §A	cost_share + conflict	strong	4, 5
budget_transfer	03: App.C§1.2; 06: Rules	check_threshold (20%)	mini	7
reporting_calendar	03: App.D§3–4; 04: §12	check_dates (rule vs asserted)	mini	8, 9, 10
report_consistency	03: App.D§4.12–4.13; 04: §12	check_dates + conflict	strong	11
gtc_version	03: Art.1.2; 00: resources	check_absent_source	mini	12
construction_read	03: Art.10, App.D§4.4	obligation lookup	mini	13

construction_vs_call	03: App.B§1.5.8, App.D§4.4; 01: refurb p.5; 02: §E; 04: 1211	callmatch + conflict	strong	21
subcontract_cap	03: App.B§1.3.2; 04: §11	check_threshold (20%)	mini	14
currency	03: App.B§2.2.5; 04: §11	method_match	mini	15
advances	03: App.B§2.2.2/2.2.4; 04: §11	check_dates + periods count	strong	16
reach_figures	03: App.A§1.1.1; 04: §5	check_sum	mini	17
indicator_value	04: §3.3 + Annex 1; 03: App.A	check_pct (recompute)	mini	18
indicator_quality	05: PMF	check_target_baseline	mini	19
partner_change	03: Art.2.2.3; 04: §10	obligation lookup	strong	20
spending_approval	04: §11; 03: App.C§1.2, App.B§2.2.6, App.D§4.4	check_threshold + approval status	strong	22
unsupported_claims	02; 04: §1, §7	grounding judgment	strong	23
cross_doc_conflict	04: cover + §1/§2; 03: App.D	conflict group	strong	24
urgency_synthesis	finding_store	score_urgency	strong	25

Unmatched questions fall through to the conflict path on the strong tier (fail safe toward flagging). After eval pass 1, the only tuning is moving an intent between mini and strong based on the escalation log.

6. Coverage map — 25 test questions

#	Topic	Step	Source docs	Code / LLM
1	Effective date + why	R→L	03 Art.3 p.3; Art.12 p.5	LLM (last signature)
2	Activity completion vs expiry	R→L	03 Art.4.1, 4.2 p.3	both

3	GAC max share; incl. overhead?	R→L	03 Art.2.1(b) p.2; App.B§1.6 p.10	LLM
4	WC min contribution + form	R→C→L	03 Art.2.2 p.2, 2.3 p.3, App.C§1.3 p.12; 02 §A	LLM + conflict
5	Does min meet call cost share?	X→C	03 Art.2.2/2.4; 01 cost share p.5	code + conflict
6	Overhead rate + base	R→L	03 App.B§1.6 p.10	LLM
7	Move 25% travel→goods?	X	03 App.C§1.2 p.12; 06 Rules	code (25% > 20%)
8	Year 3 report due; on time?	X→C	03 App.D§4.9 p.15; 04 cover p.1 & exec p.2	code + conflict
9	Year 4 work plan due	X→C	03 App.D§4.3 p.15; 04 §12 p.8	code (rule vs telephone)
10	Disposal of Assets Plan due	X→C	03 App.D§4.11 p.16; 04 §12 p.8	code + conflict
11	Final report due; consistent?	R→X→C	03 App.D§4.12–4.13 p.16; 04 §12 p.8	code + conflict
12	Which GTC version	R→L	03 Art.1.2 p.2; 00 resources p.2	LLM + absent-source
13	Required before construction	R→L	03 App.D§4.4 p.15; Art.10 pp.4–5	LLM
14	Subcontracting cap	R	03 App.B§1.3.2 p.9	LLM (+ code verify)
15	FX conversion rule	R→C	03 App.B§2.2.5 p.11; 04 §11 p.7	LLM + conflict
16	Advance periods; within rule?	R→X→L	03 App.B§2.2.4 p.11; 04 §11 p.8	both
17	Direct reach target; add up?	X	03 App.A§1.1.1 p.6; 04 §5 p.5	code (sum)
18	1210.3 HCF count + %	X	04 §3.3 p.4 & Annex 1 p.9; 03 App.A	code (% recompute)
19	No target; target < baseline	X	05 PMF (1220.1; 1110.1)	code (compare)

20	Partner change notified?	R→L	03 Art.2.2.3 p.8; 04 §10 p.7	LLM
21	Construction vs call rules	R→L→C	01 refurb p.5; 02 §E p.2; 03 App.B§1.5.8 p.10; 04 §6 p.6	LLM + checklist
22	Spending needing approval	X→L	04 §11 pp.7–8; 03 App.C§1.2, App.B§2.2.6, App.D§4.4	both
23	Claims with no evidence	R→L	02 pp.3,6; 04 §1 p.2, §7 p.6	LLM
24	Docs disagree on Y3 period/dates	C	04 cover p.1 vs exec p.2; 03 App.D p.15	conflict
25	Three most urgent issues	X→L	all 03/04/05 findings	code (urgency) + LLM

7. Deterministic checks (the oracles)

- `check_dates` — rule_date from anchor+offset vs asserted_date; status vs AS_OF (30 Jun 2022). Anchors: Effective Date 21 Jun 2019; project year Apr 1–Mar 31 (App.D §3); activities end 31 Mar 2023; expiry 30 Sep 2023.
- `check_sum` — components vs stated total (reach figures).
- `check_pct` — numerator/denominator recomputed vs stated % (denominator from App.A).
- `check_threshold` — value vs limit (20% transfer, 20% subcontract, 95% advances).
- `check_cost_share` — 90.5% / 9.5% / 10% comparisons (and the 9.5-vs-10 gap).
- `check_target_baseline` — null target; target below baseline.
- `check_absent_source` — is the answer in a known-absent doc? → abstain with metadata.
- `score_urgency` — rule-based ranking for Q25.

Where a check is the oracle (Q5, 7, 17, 18, 19), the model never owns the number — it narrates the code's result.

8. Prompt design per LLM call

Call	Tier	Output shape	Main failure to guard
Router (or rules)	mini	{intent, doc_ids, checks}	Over-narrowing docs → may add, never remove

Ledger extraction	mini	[{category, rule_verbatim, anchor, offset_days, recurring, ref}]	Inventing dates (never computes); absorbing GTC it can't see → source_absent flag
Precedence extraction	mini	ordered [{rank, part, doc_id, page, quote}]	Using precedence to resolve a conflict → context only
Call checklist	mini	[{requirement, verbatim, page}]	Paraphrasing numbers (10%, 25%, $2–20M) → verbatim required
Draft findings	strong	Finding per schema (status, evidence[], conflict, computed)	Resolving conflicts instead of flagging; fabricating quotes → verifier downgrades to needs_review
Radar summaries	mini	{summary, relevance, why, url, retrieved_at}	Inventing deadlines; stale URLs → extractive only

9. One-day implementation plan (4–5 people)

Workstreams

- **W1 Core/Ingest** — units+locators, registry (incl. known-absent), AS_OF, finding schema, quote verifier.
- **W2 Knowledge extraction** — obligations ledger, PMF table, report indicator table, call checklist (all with verbatim quotes).
- **W3 Deterministic checks** — date engine, sums, %, thresholds, PMF↔report join, target-vs-baseline.
- **W4 Reasoning + Q&A + UI** — draft-findings prompt, conflict grouping + resolver, Q&A box, dashboards, clickable citations.
- **W5 Eval + glue + lead** — hand-written gold file (25), scorer (5 categories), hallucination rate, demo script.

Checkpoints

Time	What must work
T+1h	All 7 files ingested to units with correct locators; AS_OF wired; quote verifier smoke-tested; GTC registered absent.
T+3h	Q&A answers single-source reads with verified citations: Q3, 6, 12, 13, 14.

T+5h	Deterministic checks live: Q7, 17, 18, 19; reporting calendar with computed-vs-asserted dates (Q8–11 drafts).
T+7h	Conflict detection + resolver naming: Q4, 5, 11, 15, 24; issues dashboard ranked (Q25). Full eval run #1.
T+8h	Fix top eval misses; final run; hallucination rate reported; demo rehearsed.

Cut list (in order)

1. Tool 3 funding radar (no test data, 2026 scenario — zero score impact).
2. Human review / correction subsystem → keep only the "unreviewed" label.
3. Tool 1 IATI / Project Browser past-project search.
4. LLM router → rules dispatch / full context.
5. Precedence auto-extraction → read Art. 1.3 once.
6. Budget-by-year + invented test-data checks → rely on report percentages.

Protected (never cut): ingest+locators, quote verifier, deterministic date/sum/% checks, conflict-flagging with resolver, gold eval + hallucination rate.

10. Test-data fixtures (budget)

File 06's Budget Summary has an **Automatic checks** block computing five ratios against limits (GAC share 0.905; WC agreement min 0.095; WC call min 0.10; subcontracting 0.20; construction 0.20). Filled copies of the workbook exercise these — but Python check functions do the math (workbook formulas = the spec), not Excel recalc.

- **F0 all-green** — every ratio within limits.
- **F1 cost-share shortfall** — WC at 9.5%: passes agreement-min, trips call-min (the Q5 story).
- **F2 single breaches** — one each for subcontracting >20% and construction ≥20%.
- **F3 baseline + revised pair** — for transfer/overspend >20% (Q7, Q22).

All fixtures labelled "TEST DATA — not from the provided documents" and tagged `test_data` so they can never surface as an answer.

11. Known-absent sources

Referenced-but-not-in-pack documents are registered so the tool abstains precisely instead of inventing content:

- **General Terms & Conditions** — incorporated by reference (03 Art.1.2 p.2), ranked above

appendices (Art.1.3), governs Survival Clause articles (Art.5 p.5), subcontracts/procurement/amendments. Answer to "which version" = in force on 21 Jun 2019.

- **Project Implementation Plan (PIP)** — due 180 days after signature (03 App.D§4.1 p.15); file 05 is its PMF, but the narrative (EMP, gender strategy, full indicator set incl. 1000.2) is absent.
- **Local Partners – Information form** — required by call (01 p.5), explicitly "not included" (02 §C p.2).
- **Reporting Forms A–D** — referenced in 03 App.D §3 and Art.1.3(d).

Do **not** fetch the GTC: the online version is the 2026 current one, not the 2019 version in force, so fetching would answer Q12 wrong. Treat any external fetch as a separate, clearly-labelled source.

12. Next buildable artifacts (dependency order)

1. **core/schema.py** — the Finding + SourceRef contract (unblocks everyone; least-churn).
2. **Ingest + quote verifier** — foundation everything reads from.
3. **Known-absent registry** — small; unblocks check_absent_source.
4. **Check-function stubs + validate()** — the oracles and the escalation gate.
5. **Gold-answer file** — hand-authored from the coverage map; no dependency on the pipeline, can start immediately.